

NumPy Array Operations

Section 1 – Question:

- Create a NumPy array of integers from 1 to 10 and:
 - find its sum
 - find its mean
 - reshape it into a 2×5 matrix

Section 2 – Answer:

```
★ import numpy as np  
arr = np.arange(1, 11)  
total_sum = np.sum(arr)
```

```
mean_value = np.mean(arr)  
reshaped_arr = arr.reshape(2, 5)
```

Section 3 – Output Explanation

• Original Array → [1 2 3 4 5 6 7 8 9 10]

• Sum → 55

• Mean → 5.5

• Reshaped Matrix → $\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \end{bmatrix}$

Numpy 2D Array Operations

Notes by
Humna Imran

Section 1 – Question:

Write a NumPy program to:

- create a 2D array
- find maximum and minimum values
- perform element-wise multiplication

Section 2 – Answer:

```
import numpy as np
array1 = np.array([[1, 2, 3],
                   [4, 5, 6]])
array2 = np.array([[10, 20, 30],
                   [40, 50, 60]])

max_val = np.max(array1)
min_val = np.min(array1)
result = np.multiply(array1, array2)
```

Section 3 – Output / Explanation

• Array 1 → $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$

• Array 2 → $\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}$

• Maximum Value → 6

• Minimum Value → 1

• Element-wise Multiplication → $\begin{bmatrix} 10 & 40 & 90 \\ 160 & 250 & 360 \end{bmatrix}$

NumPy Array Generation & Statistics

Notes by
Humna Imran

Section 1 – Question:

Using NumPy:

- generate an array using arange() or linspace()
- perform slicing
- calculate standard deviation

Section 2 – Answer:

```
import numpy as np
arr = np.linspace(0, 10, 11)
subset = arr[2:6]
std_dev = np.std(arr)
```

Section 3 – Output/Explanation

- Original Array → [0. 1. 2. 3. 4. 5. 6. 7. 8. 9. 10.]
- Sliced Array (index 2 to 5) → [2. 3. 4. 5.]
- Standard Deviation → 3.16

Pandas DataFrame Operations

Notes by Humna Imran

Section 1 – Question:

Create a Pandas DataFrame with student names and marks and:

- display first 5 records
- calculate average marks
- filter students with marks greater than a given value

Section 2 – Answer:

```
import pandas as pd
```

1. Create the DataFrame

```
data = {  
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva', 'Frank', 'Grace'],  
    'Marks': [85, 92, 78, 60, 88, 75, 95, 82]  
}  
df = pd.DataFrame(data)
```

2. Display first 5 records

```
print('--- First 5 Records ---')  
print(df.head())
```

3. Calculate average marks

```
average_marks = df['Marks'].mean()  
print(f'\nAverage Marks: {average_marks:.2f}')
```

4. Filter students with marks greater than 80

```
threshold = 80  
high_scorers = df[df['Marks'] > threshold]  
print(f'\n--- Students with marks > {threshold} ---')  
print(high_scorers)
```


Pandas CSV Handling & Missing Values

Section 1 – Question:

Write a Pandas program to:

- read a CSV file
- display basic information using info() and describe()
- handle missing values

Section 2 – Answer:

```
import pandas as pd
import numpy as np
--- Step 0: Create a dummy CSV file for this example ---
data = {
    'Student_ID': [101, 102, 103, 104, 105],
    'Subject': ['AI', 'Data Science', 'Networking', 'AI', 'Calculus'],
    'Marks': [85, np.nan, 78, 92, np.nan] # np.nan represents missing data
}
Saving this to a file named 'class_data.csv'
pd.DataFrame(data).to_csv('class_data.csv', index=False)
```

Main Program Starts Here

1. Read a CSV file

```
df = pd.read_csv('class_data.csv')
print('Original Data ')
print(df)
```

2. Display basic information

```
print('\n Data Info (df.info()) ')
info() tells you about data types and how many non-null values exist
df.info()
print('\n Statistical Summary (df.describe()) ')
describe() gives you count, mean, std, min, max for numerical columns
print(df.describe())
```

3. Handle missing values

Strategy: Fill missing marks with the average (Mean Imputation)

```
average_mark = df['Marks'].mean()
df['Marks'] = df['Marks'].fillna(average_mark)
print(f'\n --- Data after filling missing values with mean ({average_mark}) ---')
print(df)
```


Pandas DataFrame Operations

Section 1 – Question:

- Create a DataFrame and demonstrate:
 - column selection
 - row filtering
 - sorting based on a column

Section 2 – Answer:

```
import pandas as pd
```

1. Create a DataFrame

```
data = {  
    'Student_ID': [101, 102, 103, 104, 105],  
    'Name': ['Ali', 'Bilal', 'Zara', 'Dawood', 'Eshal'],  
    'Major': ['CS', 'AI', 'SE', 'AI', 'CS'],  
    'GPA': [3.2, 3.8, 2.9, 3.5, 3.9]  
}  
df = pd.DataFrame(data)  
print(' Original DataFrame ')  
print(df)
```

2. Column Selection

Select specific columns (returns a new DataFrame)

```
selected_columns = df[['Name', 'GPA']]  
print('\nSelected Columns (Name & GPA) ')  
print(selected_columns)
```

3. Row Filtering

Filter for students with a GPA greater than 3.4

```
high_achievers = df[df['GPA'] > 3.4]  
print('\n Filtered Rows (GPA > 3.4) ')  
print(high_achievers)
```

4. Sorting based on a column

Sort by GPA in descending order (highest to lowest)

```
sorted_df = df.sort_values(by='GPA', ascending=False)  
print('\n Sorted by GPA (Descending) ')  
print(sorted_df)
```


Pandas-NumPy Integration

Section 1 – Question:

Create a Pandas DataFrame and:

- convert one column into a NumPy array
- perform mathematical operations using NumPy

Section 2 – Answer:

```
import pandas as pd
import numpy as np
```

1. Create a Pandas DataFrame

```
data = {
    'Product': ['Widget A', 'Widget B', 'Widget C', 'Widget D'],
    'Price': [100, 250, 400, 150],
    'Quantity': [5, 2, 8, 10]
}
df = pd.DataFrame(data)
print(' Original DataFrame ')
print(df)
```

2. Convert one column ('Price') into a NumPy array

`.to_numpy()` is the recommended method over `.values`

```
price_array = df['Price'].to_numpy()
print(f'\nExtracted NumPy Array: {price_array}')
print(f'Type: {type(price_array)}')
```

3. Perform mathematical operations using NumPy

★ Example A: Apply a flat discount of 10% to all prices

```
discounted_prices = price_array * 0.90
```

Example B: Calculate total revenue (Price * Quantity) using vectorization
We extract Quantity as well to do array-to-array multiplication

```
quantity_array = df['Quantity'].to_numpy()
```

```
total_revenue = np.sum(price_array * quantity_array)
```

```
print('\n NumPy Operations Results ')
```

```
print(f'Discounted Prices (10% off): {discounted_prices}')
```

```
print(f'Total Revenue Calculation: {total_revenue}')
```


NumPy & Its Importance in Artificial Intelligence

Question:

Explain NumPy and its importance in Artificial Intelligence.

Answer:

NumPy (Numerical Python) is the fundamental package for scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of high-level mathematical functions to operate on these arrays. In the world of Artificial Intelligence (AI) and Data Science, NumPy is often considered the "foundation" upon which almost all other tools are built. Here is why NumPy is indispensable for AI:

1. Speed and Efficiency (The "Heavy Lifting")

↳ Python is an interpreted language, which makes it easy to write but generally slow for heavy calculations. AI algorithms often require millions of mathematical operations (like matrix multiplications) to train a single model.

- **Contiguous Memory:** Unlike standard Python lists, which are scattered in memory, NumPy arrays are stored in contiguous blocks of memory. This allows the CPU to process data much faster (SIMD – Single Instruction, Multiple Data).
- **C-Integration:** Most of NumPy's core logic is written in optimized C. When you run a NumPy command, you get the ease of Python but the speed of C.

2. N-Dimensional Arrays (Tensors)

↳ In AI, data is rarely just a single number. It is almost always a structure. NumPy's ndarray (N-dimensional array) is the standard way to represent this data.

- **1D Array:** Audio waveforms or time-series data.
- **2D Array:** Excel spreadsheets or grayscale images.
- **3D Array:** Color images (Height × Width × 3 RGB Channels).
- **4D Array:** A batch of video frames or MRI scans.

3. Vectorization and Broadcasting

↳ AI code needs to be clean and fast. NumPy allows for **Vectorization**—applying operations to whole arrays at once without writing explicit for loops.



Without NumPy (Slow Loop):

```
Python
# Multiply every number by 2
result = []
for x in data:
    result.append(x * 2)
```

With NumPy (Vectorized):

```
Python
result = data * 2 # Happens instantly across
millions of numbers
```

4. Linear Algebra Capability

↳ At its core, Deep Learning is **Linear Algebra**. Neural Networks learn by adjusting “weights” (matrices) via **dot products and gradients**.

comes with a built-in linear algebra engine (**numpy.linalg**) capable of calculating:

- **Dot products (Matrix Multiplication)**
- **Eigenvalues and Eigenvectors**
- **Matrix inversions**

* Pandas: Series & DataFrame

Question:

What is Pandas? Explain Series and DataFrame with differences.

→ Answer:

Pandas is an open-source Python library built on top of NumPy. It is the industry standard for data manipulation and analysis. If NumPy provides the “engine” for high-performance calculations, Pandas provides the “dashboard” — a user-friendly interface to structure, clean, and analyze messy real-world data (like CSVs, Excel files, and SQL databases).

• Core Components: Series vs. DataFrame

Pandas is built around two primary data structures: the Series and the DataFrame.

1. Series (1-Dimensional)

A Series is essentially a single column of data. It is very similar to a NumPy array or a list, but with one key difference: it has an index (labels) associated with each data point.

Analogy: Think of a Series as a single column in an Excel sheet.

Structure: Index + 1 Column of Data.

```
import pandas as pd
```

```
# Creating a Series
```

```
ages = pd.Series([22, 35, 58], index=['Alice', 'Bob', 'Charlie'])  
print(ages)
```

Output: →

Alice	22
Bob	35
Charlie	58
dtype: int64	

2. DataFrame (2-Dimensional)

A DataFrame is a table.

A DataFrame is a table. It is a collection of multiple Series that share the same index. This is the most common object you will work with in Data Science.

Analogy: Think of a DataFrame as the entire Excel spreadsheet (rows and columns).

Structure: Index (rows) + Columns (headers) + Data.

Creating a DataFrame

```
data = {  
    'Age': [22, 35, 58],  
    'City': ['New York', 'Paris', 'London']  
}
```

```
df = pd.DataFrame(data, index=['Alice', 'Bob', 'Charlie'])  
print(df)
```

Output: →

	Age	City
Alice	22	New York
Bob	35	Paris
Charlie	58	London

Key Differences at a Glance

Feature	Series	DataFrame
Dimensions	1D (One dimension)	2D (Rows and Columns)
Data Content	Holds data of a single data type (mostly).	Can hold different data types in different columns (e.g., integers in Col A, strings in Col B).
Visual	A single list/column.	A full table/spreadsheet.
Access	Accessed by a single index label (e.g., s['Alice']).	Accessed by row index AND column name (e.g., df['Age']['Alice']).

In short: A DataFrame is just a container for one or more Series. When you pull a single column out of a DataFrame, it actually becomes a Series!

Difference Between NumPy and Pandas

Question:

Differentiate between:

- NumPy and Pandas

Answer:

1. Data Structure & Types

This is the most fundamental difference.

→ NumPy (Homogeneous):

NumPy arrays are homogeneous, meaning every single element in the array must be of the same data type (e.g., all integers or all floats). Because they are uniform, they are stored in contiguous blocks of memory, making them incredibly fast for processing.

Best for: Matrices, images (pixels), and raw numerical data.

→ Pandas (Heterogeneous):

Pandas DataFrames are heterogeneous. Each column can have a different data type. You can have a column for "Names" (strings), a column for "Age" (integers), and a column for "Salary" (floats) all in one table.

Best for: Real-world tabular data (like CSVs, Excel sheets, SQL tables).

2. Indexing

NumPy: Uses implicit integer-based indexing (0, 1, 2...). You access data by its position.

Pandas: Uses explicit label-based indexing. While you can use numbers, you primarily access data by column names (`df['Age']`) or row labels (`df.loc['row_name']`).

3. Comparison Table

Feature	NumPy (np)	Pandas (pd)
Primary Object	ndarray (N-dimensional array)	DataFrame (Table) & Series (Column)
Performance	Extremely fast; optimized for calculation.	Slower than NumPy; optimized for data handling.
Memory Usage	Consumes less memory.	Consumes more memory (overhead for index/headers).
Math Focus	Linear Algebra, Fourier Transforms, Matrix Math.	Statistics, Aggregation, Time Series Analysis.
Relationship	The Base Layer.	Built on top of NumPy.

4. Code Comparison

NumPy Way (Mathematical):

```
import numpy as np
arr = np.array([1, 2, 3])
print(arr / 2)
# Output: [0.5  1.  1.5]
```

→ Pandas Way (Analytical):

```
import pandas as pd
s = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
print(s.describe())
# Output gives count, mean, std, min, max, etc.
```

Summary-

- If you are building a Neural Network from scratch, you use NumPy for the matrix multiplication.
- If you are loading a messy Excel file to find the average sales per region, you use Pandas.

Question: Difference between Matplotlib and Seaborn

Answer:

Matplotlib and Seaborn are the two titans of Python data visualization. While they are often used together, they serve different roles: Matplotlib is the "engine," while Seaborn is the "luxury car" built on top of it. Here is the detailed breakdown of their differences.

→ 1. The Core Relationship

- **Matplotlib** is a low-level library. It gives you complete control over every single pixel, line, and label, but you often have to write a lot of code to get there.
- **Seaborn** is a high-level library built on top of Matplotlib. It simplifies complex plots into single lines of code and comes with beautiful default styles.

Think of it this way:

- **Matplotlib** is like buying individual ingredients (flour, sugar, eggs) to bake a cake.
- **Seaborn** is like buying a high-quality cake mix—you get a great result much faster, but you have slightly less control over the specific chemistry.

→ 2. Key Differences Table

Feature	Matplotlib	Seaborn
Code Style	Verbose: You often need multiple lines to set titles, labels, and legends manually.	Concise: A single function often handles the plot, labels, colors, and legends automatically.
Aesthetics	Basic: The default plots can look "flat" or "academic." You must manually style them to make them look modern.	Modern: Comes with built-in themes (e.g., darkgrid) and sophisticated color palettes that look professional instantly.
Data Handling	Works best with NumPy arrays or lists. You have to manually separate data for X and Y axes.	Built specifically for Pandas DataFrames. It understands column names and can split/group data automatically.
Specialty	Basic 2D/3D plotting (Lines, Bars, Scatters).	Statistical plotting (Heatmaps, Violin plots, Regression lines).

Difference between Matplotlib and Seaborn

3. Syntax Comparison

Let's say we want to create a scatter plot and color the dots based on a category (e.g., "Male" vs "Female").

Matplotlib (The "Manual" Way):

You have to manually loop through the categories or filter the data yourself to assign colors.

```
import matplotlib.pyplot as plt
```

You often have to filter data manually

```
plt.scatter(male_data['x'], male_data['y'], color='blue', label='Male')  
plt.scatter(female_data['x'], female_data['y'], color='red', label='Female')  
plt.legend()  
plt.show()
```

Seaborn (The "Smart" Way):

Seaborn handles the coloring and legend automatically using the hue parameter.

```
import seaborn as sns
```

Pass the DataFrame and just name the columns

```
sns.scatterplot(data=df, x='total_bill', y='tip', hue='sex')
```

4. When to Use Which?

• Use Seaborn when:

- You are doing Exploratory Data Analysis (EDA).
- You are working with Pandas DataFrames.
- You need complex statistical plots like Heatmaps, Violin Plots, or Pair Plots (grids of plots).
- You want a good-looking chart in 10 seconds.

• Use Matplotlib when:

- You need 3D plots (Seaborn doesn't support 3D).
- You are building a totally custom chart type from scratch.
- You need to tweak a very specific element (e.g., "Rotate the text on the Y-axis by 45 degrees").

Note:

You often import Matplotlib alongside Seaborn just to tweak the final details of a Seaborn plot.

Matplotlib & Data Visualization

Question:

Explain the role of Matplotlib in data visualization.

Answer:

Matplotlib is the foundational library for data visualization in Python. It is a multi-platform data visualization library built on NumPy arrays and designed to work with the broader SciPy stack.

In the context of AI and Data Science, its role is comparable to the "canvas and paint" of an artist. It provides the raw tools necessary to render almost any type of 2D plot (and basic 3D plots).

Here is a breakdown of its specific roles:

1. The Anatomy of a Plot

Matplotlib allows you to control every aspect of a chart. To understand its role, you must understand its hierarchy:

- Figure: The entire window or page that everything is drawn on.
- Axes: The specific plot region (where the x and y axes are). You can have multiple Axes in one Figure (subplots).
- Axis: The number lines (x-axis and y-axis) that set the scale.

2. Exploratory Data Analysis (EDA)

Before training any AI model, you must understand your data. Matplotlib is the primary tool used to:

- Identify Outliers: Using scatter plots to spot data points that don't fit the pattern.
- Check Distributions: Using histograms to see if data is normal (Gaussian) or skewed.
- Correlation: Visualizing relationships between variables (e.g., Study Hours vs. Grades).

3. Model Evaluation (Crucial for AI)

In Artificial Intelligence, Matplotlib is essential for visualizing how well a model is learning.

- Loss Curves: Plotting "Training Loss" vs. "Validation Loss" over epochs to detect overfitting or underfitting.
- Decision Boundaries: Visualizing how a classifier separates different classes of data.

4. Image Visualization

Since images are essentially NumPy arrays of pixels, Matplotlib is the standard tool for displaying them in notebooks.

- `plt.imshow()`: This function is used constantly in Computer Vision to display input images, feature maps, or generated outputs.

Matplotlib in Data Visualization

Backend for Other Libraries

Many high-level libraries (like Seaborn and Pandas plotting) are wrappers around Matplotlib.

When you run `df.plot()` in Pandas, it is actually using Matplotlib to draw the lines.

Matplotlib provides the low-level engine that powers these easier-to-use tools.

Code Example: A Simple AI-Related Plot

Here is how you might visualize the "Loss" of a model decreasing over time:

```
import matplotlib.pyplot as plt

import numpy as np
1. Simulate data (Epochs vs Loss)
   epochs = np.arange(1, 11)

   training_loss = [0.9, 0.7, 0.5, 0.4, 0.3, 0.25, 0.2, 0.15, 0.12, 0.1]
2. Create the Plot
   plt.figure(figsize=(8, 5)) # Set the figure size

   plt.plot(epochs, training_loss, marker='o', color='red', linestyle='--')
3. Add Details (The "Matplotlib" control)
   plt.title('Model Training Loss')

   plt.xlabel('Epochs')

   plt.ylabel('Loss')

   plt.grid(True) # Add a background grid
4. Show the plot
   plt.show()
```

Summary

Matplotlib is not always the fastest way to get a pretty chart (Seaborn is faster for that), but it is the most powerful because it allows you to customize every single tick, line, and color. It is the industry standard for academic and technical plotting.

Seaborn vs Matplotlib

Notes by Humna Imran

Question: What is Seaborn and how is it different from Matplotlib?

Answer:

Seaborn is a high-level Python data visualization library based on Matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.

While Matplotlib provides the raw building blocks for creating charts, Seaborn provides pre-assembled "blueprints" for complex statistical plots, making it easier to visualize relationships in your data. It is tightly integrated with Pandas DataFrames, making it the go-to tool for Data Science analysis.

Here is how it differs from Matplotlib:

1. Level of Abstraction (The "Stick Shift vs. Automatic" Analogy)

Matplotlib (Low-Level): It is like driving a manual car. You have total control over the gears (pixels, lines, ticks), but you have to do a lot of manual work to get from point A to point B.

→ **Seaborn (High-Level):** It is like driving an automatic luxury car. It handles the shifting for you (colors, scales, aggregation) so you can focus on the destination (the data insights).

2. Integration with Pandas

Matplotlib: Primarily designed to work with NumPy arrays or Python lists. To plot data from a Pandas DataFrame, you often have to extract the columns as arrays first or write loops to separate categories.

→ **Seaborn:** Built specifically to understand Pandas DataFrames. You can simply pass the DataFrame and the column names (e.g., `x='Age'`, `y='Salary'`), and Seaborn automatically handles the mapping.

3. Statistical Functionality

Seaborn is not just for drawing; it is for calculating. It performs statistical aggregation automatically.

Matplotlib: If you want a bar chart showing the average salary, you must calculate the mean yourself first, then pass that number to the plot function.

→ **Seaborn:** You just pass the raw data. Seaborn automatically calculates the mean and draws the bar, and it will even add error bars (confidence intervals) by default to show the reliability of the data.

4. Aesthetics and Themes

Matplotlib: Default plots often look "academic" or basic (standard blue lines on a white background). You have to write extra code to add gridlines, change palettes, or style the background.

→ **Seaborn:** Comes with built-in themes (like darkgrid, whitegrid) and sophisticated color palettes (e.g., for heatmaps) that make plots look publication-ready instantly.

Summary Table

Feature	Matplotlib	Seaborn
Foundation	The Base Library.	Built on top of Matplotlib.
Syntax	Verbose (More code).	Concise (Less code).
Data Input	Arrays / Lists.	Pandas DataFrames.
Best For	Custom plotting, 3D plots, basic charts.	Statistical analysis, Heatmaps, Exploratory Data Analysis.

Example: Creating a Heatmap

This is where the difference is most obvious. Creating a correlation heatmap in Matplotlib requires significant manual setup. In Seaborn, it is one line:

```
import seaborn as sns
import matplotlib.pyplot as plt
# Load a sample dataset
df = sns.load_dataset("tips")
# Create a correlation matrix
corr = df.corr(numeric_only=True)
# Seaborn: One line to draw a complex heatmap with colors and
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.show()
```